

OpenStack Advanced

Part 2 of 2

*Images, Object Storage
& HPC Integration*

Cloud Computing

ADSAI – University of Trieste

Academic Year 2025/2026

Topics: Glance image lifecycle • Custom HPC images • NFS shared filesystem • Swift object storage • HPC-cloud integration patterns • Cloud bursting • Troubleshooting • Best practices

Builds on: Part 1 (IaC, Heat, Terraform) • L04 (Docker images) • L02 (Cloud economics)

Image Management with Glance: Why Custom Images?

Install once, boot many times — eliminate 10–15 min of apt-get on every VM launch

Problem: Vanilla Cloud Images

- Vanilla cloud images are minimal and generic — no HPC software
- Every launch = 10–15 min of apt-get install + compile steps
- N workers × same download = wasted bandwidth on every run
- Package versions drift between launches — non-reproducible
- Unscientific: environment details not captured alongside results

Solution: Custom Glance Images

- Build image once: install, GCC, BLAS, MPI, Python, custom tools
- Snapshot → upload to Glance → boot N instances in ~90 seconds
- Versioned: Ubuntu-22.04-HPC-v1.2
- Build scripts in Git → anyone can rebuild the exact environment
- AWS analog: AMI (Amazon Machine Image) — identical concept

Principle: *install once, boot many times.*

A well-designed image is an immutable, versioned artifact — just like a Docker image (L04). Environment reproducibility is a scientific requirement, not an optional convenience.

Glance (OpenStack) ↔ AMI (AWS EC2) ↔ Docker image (containers): same 'immutable versioned environment' pattern at different abstraction levels

Building Custom Images: Snapshot + Upload Approaches

Approach A: snapshot running VM | Approach B: upload pre-built image

Approach A: Snapshot a Running VM

```
# 1. Launch builder VM from base image
openstack server create \
  --image 'Ubuntu 22.04' --flavor m1.medium \
  --key-name my-key --network my-net image-builder

# 2. SSH in and install HPC software
ssh ubuntu@<floating-ip>
sudo apt-get install -y openmpi-bin libopenmpi-dev \
  python3-numpy python3-scipy htop nfs-common
# Compile OPENGADGET, LAMMPS, etc.

# 3. Clean before snapshot (important!)
sudo apt-get clean
sudo cloud-init clean --logs
sudo rm -rf /tmp/* /var/tmp/*

# 4. Stop and create image
openstack server stop image-builder
openstack server image create \
  --name 'Ubuntu-22.04-HPC-v1.0' image-builder
# Monitor status (may take several minutes)
openstack image list --name 'Ubuntu-22.04-HPC-v1.0'
```

Approach B: Upload Pre-built Image

```
wget https://cloud-images.ubuntu.com/jammy/\
  current/jammy-server-cloudimg-amd64.img

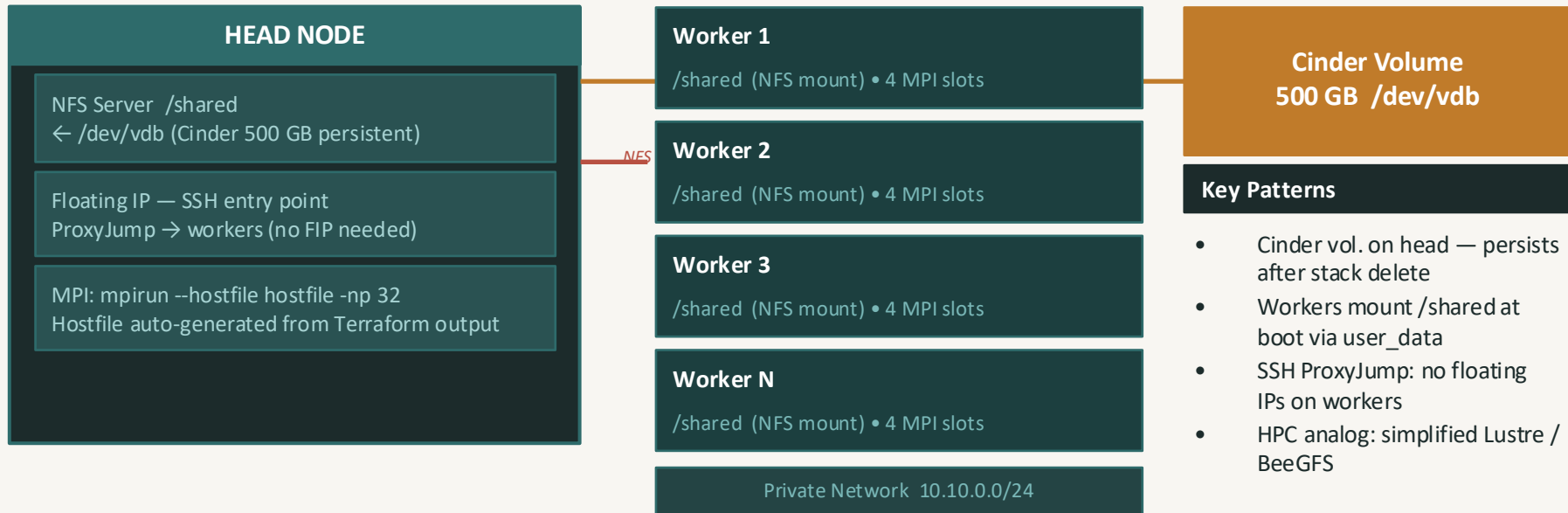
openstack image create \
  --file jammy-server-cloudimg-amd64.img \
  --disk-format qcow2 \
  --container-format bare \
  --min-ram 512 --min-disk 3 \
  'Ubuntu-22.04-Official'
```

Lifecycle Best Practices

Practice	Rationale
Semantic name versioning	Ubuntu-22.04-HPC-v1.2
Document in Glance metadata	openstack image set --property installed_sw=...
Store build scripts in Git	Reproducible anyone can rebuild
Test before marking official	Boot test VM + smoke test first
Packer for automation	Eliminates manual steps; enables CI/CD

Multi-Instance Clusters: NFS Shared Filesystem

Cinder-backed volume on head node exported via NFS to workers



HPC note: NFS suffers from high metadata overhead for many small files. For large simulations, consider dedicated Cinder volumes per worker. Use NFS for shared parameter files and checkpoints, not hot scratch data.

Swift: OpenStack Object Storage ($\approx$ AWS S3)

Eventual consistency, RESTful HTTP, containers + objects — durable data exchange layer

Account	Container	Object	Temp URL
Top-level namespace. One per project. AUTH_<project-id>.	Like an S3 bucket. Flat — groups objects. No subfolder nesting.	Arbitrary blob + metadata. URL-addressable. Max 5 GB (SLO for >5 GB).	Time-limited pre-signed URL — share without full auth. $\approx$ S3 pre-signed.

Essential CLI Commands

```
openstack container create sim-inputs sim-outputs

# Upload individual objects
openstack object create sim-inputs reference-genome.fa
openstack object create sim-inputs params.json

# Upload directory (python-swiftclient)
pip install python-swiftclient
swift upload sim-inputs ./input-data/

# Download object
openstack object save sim-outputs results-run42.tar.gz

# Temporary URL (3600 s = 1 hour)
openstack object store account set \
  --property Temp-URL-Key=<secret>
swift tempurl GET 3600 \
  /v1/AUTH_<proj>/sim-inputs/data.tar.gz <secret>
```

Swift: OpenStack Object Storage ($\approx$ AWS S3)

Eventual consistency, RESTful HTTP, containers + objects — durable data exchange layer

Account	Container	Object	Temp URL
Top-level namespace. One per project. AUTH_<project-id>.	Like an S3 bucket. Flat — groups objects. No subfolder nesting.	Arbitrary blob + metadata. URL-addressable. Max 5 GB (SLO for >5 GB).	Time-limited pre-signed URL — share without full auth. $\approx$ S3 pre-signed.

Swift for HPC Data Staging	
Use Case	Swift Pattern
Pre-stage inputs	Upload ref DBs + param files before launching VMs
Checkpointing	Workers write sim checkpoints periodically
Results archival	Final outputs persist after stack deletion
Data sharing	Temp URLs for collaborators — no full OS access
HPC $\rightarrow$ Cloud	swift download on ADA; input data from Leonardo
Cloud $\rightarrow$ HPC	swift download on HPC; avoid re-running prep steps

HPC-Cloud Integration: Three Complementary Patterns

ADA Cloud and Leonardo/Galileo serve different roles — use each for what it does best

Interactive Analysis

ADA Cloud as interactive compute environment

1

CINECA HPC (Leonardo): batch-optimised, strict policies on interactive use.
ADA Cloud: Jupyter Lab, RStudio, Dash — unrestricted interactive sessions.
Workflow: prototype + explore on ADA → submit production jobs to HPC SLURM.
Cloud VMs persist while you work; HPC nodes released when job ends.

Pre/Post-Processing on Cloud

Asymmetric phases: cheap VM work flanks expensive HPC runs

2

Pre-processing on ADA: data cleaning, format conversion, parameter sweep generation.
Main simulation on HPC: MPI-parallel, GPU, InfiniBand — where ADA cannot compete.
Post-processing on ADA: statistical analysis, publication-quality visualisation.
Data transfer: ADA ↔ HPC via Swift object storage or SFTP.

Cloud Bursting

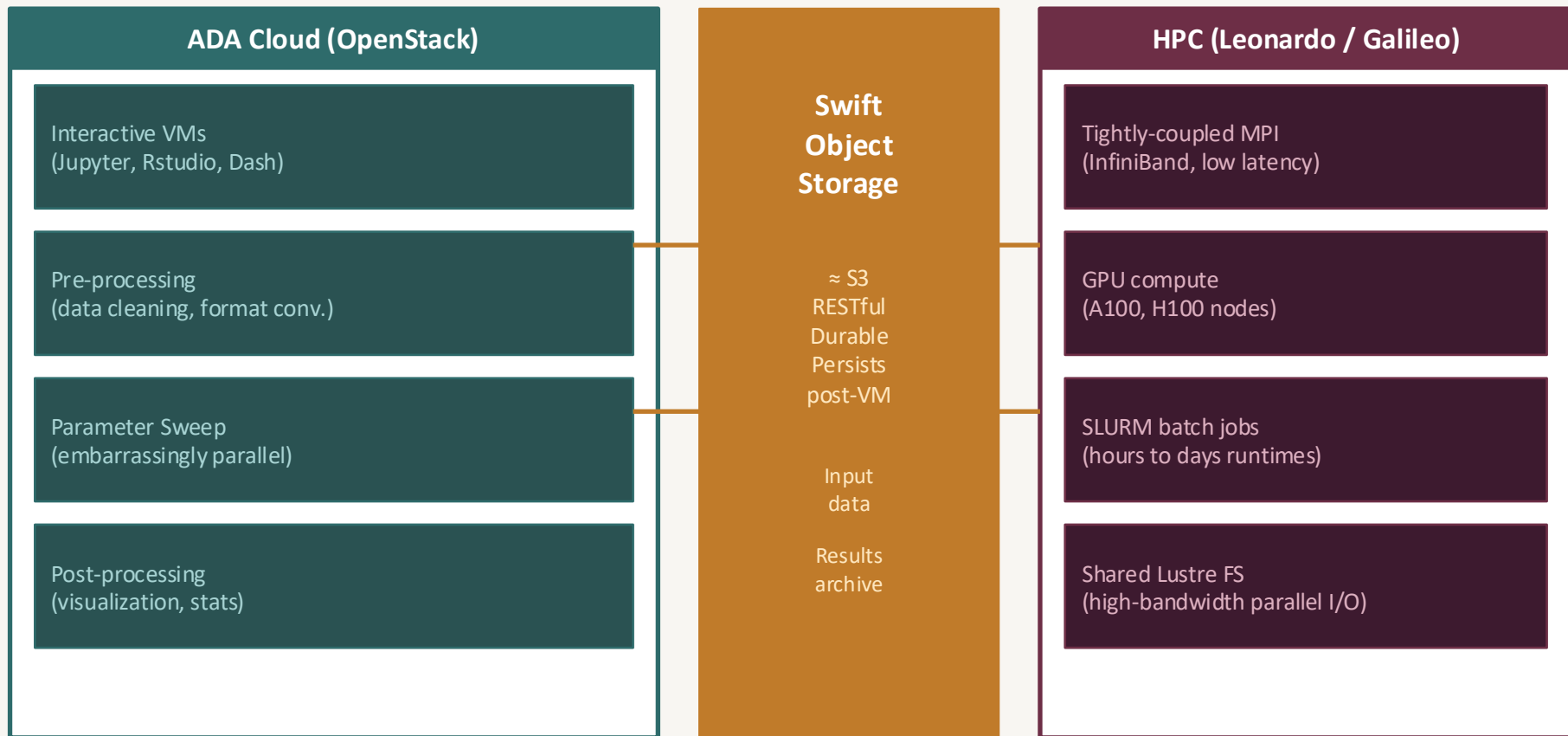
Overflow embarrassingly parallel workloads when HPC queues are long

3

Only works for embarrassingly parallel tasks: parameter sweeps, independent replicas.
Cannot burst tightly-coupled MPI (no InfiniBand between ADA and HPC).
Pattern: submit primary job to SLURM; simultaneously burst overflow to ADA.
terraform destroy when sweep completes — zero idle compute cost.

Hybrid HPC-Cloud Architecture

Swift is the neutral data exchange layer — ADA and HPC never communicate directly



Troubleshooting: Common Failure Modes and Diagnosis

Systematic approach — check quotas first, then state, then console log

Symptom	Likely Cause	Diagnostic Command	Resolution
Auth error (401)	Invalid/expired credentials	openstack token issue	Re-source openrc; regenerate App Credential
Instance stuck in BUILD	Scheduler cannot find host	openstack server show <vm>	Check quotas; try a smaller flavor
Instance in ERROR state	Boot failure in cloud-init	openstack console log show <vm>	Fix user_data; check cloud-init output
Cannot ping instance	Missing ICMP rule in SG	openstack security group rule list	Add ICMP ingress rule to security group
Cannot SSH to instance	No SSH rule or no FIP	openstack floating ip list	Check SG rule (tcp/22) + FIP assignment
Volume attach fails	Volume/instance in different AZ	openstack volume show <vol>	Create volume in same AZ as instance
Floating IP alloc fails	External IP pool exhausted	openstack quota show	Release unused FIPs or contact CINECA admin
Heat CREATE_FAILED	Resource dependency error	openstack stack failures list <n>	Inspect failed resource; fix template
terraform apply fails	State mismatch or API error	terraform plan	Assess drift; check terraform state list

Systematic Debug Procedure: Five Steps

Applicable to any failed OpenStack deployment

①

Check Quotas First

```
openstack quota show | openstack limits show --absolute
```

Quota exhaustion causes silent 403 failures. Most common cause of mysterious resource-creation failures.

②

Inspect Stack/State

```
openstack stack failures list <n> | terraform plan
```

Identify WHICH resource failed and the exact error message. For Terraform: terraform show inspects the state.

③

Console Log

```
openstack console log show <vm> | tail -80
```

Cloud-init output is here. Reveals user_data script errors, boot failures, package install problems.

④

Network Check

```
openstack port list --server <vm> | openstack floating ip list
```

Verify floating IP assignment. Check security group rules for the specific protocol/port.

⑤

noVNC Console

```
openstack console url show --type novnc <vm>
```

Interactive terminal in browser — last resort when SSH is unreachable. Useful for boot-level diagnostics.

Best Practices: IaC Discipline and Authentication Security

Research-grade deployments require more than working code

Rule: Never provision research infrastructure manually if results will be published. Every VM, volume, network, and security group must be in a Heat/Terraform config committed to version control alongside code and data.

Authentication Security

Practice	Rationale
Application Credentials, never passwords	Scoped, revocable; don't expose user credentials
Credentials in env vars, not hardcoded files	Prevents accidental commit to version control
Add *.openrc*.sh to .gitignore	Ensure credentials never reach a public repository
Rotate App Credentials periodically	Limits exposure window if credential is compromised
Separate credentials per project	Limits blast radius of any single compromise

Resource Tagging

```
openstack server set \  
  --property owner=$USER \  
  --property project=genome-analysis \  
  my-instance  
openstack volume set \  
  --property purpose=simulation-data research-vol
```

Quota Hygiene on CINECA ADA

- No per-hour billing on ADA — but quotas are shared. Treat as shared resource.
- Always terraform destroy / openstack stack delete when work is done.
- Use smallest flavor during development; scale up only for production runs.
- Monitor: openstack limits show --absolute before each large deployment.

Best Practices: Data Durability Strategy

Match storage tier to data lifecycle — not all data survives VM deletion

Data Category	Storage Strategy	Durability	Survives VM delete?
Input datasets	Cinder vol on head; backup to Swift	High	Yes (Cinder) / Yes (Swift)
Active simulation state	Cinder vol; periodic checkpoints to Swift	Medium	Yes (Cinder) / Yes (Swift)
Scratch / temporary	Ephemeral disk or tmpfs	None	No — lost on VM delete
Results / outputs	Swift (primary) + Cinder backup	High	Yes — Swift survives stack delete
Code & configuration	Git repository (essential)	High	Yes — independent of cloud
VM configuration	Heat HOT / Terraform state in Git	High	Yes — reproducible from scratch

Three Golden Rules

1

Never store results only in VM ephemeral disk

The VM will be deleted. The results will be gone.

2

Swift is the durable data layer between ADA and HPC

Persists independent of VM and stack lifecycle.

3

Code + IaC config in Git is your reconstruction guarantee

Given Git, recreate the entire environment from scratch.

End-to-End Research Workflow on CINECA ADA

From raw data to published results — integrating all concepts from both lectures

1

Prepare & Version

Write HOT / Terraform config. Commit to Git. Source openrc.sh.

2

Build Custom Image

Install APPS + MPI. cloud-init clean. openstack server image create.

3

Stage Input Data

openstack container create; swift upload → input data available in object store.

4

Deploy Cluster

terraform apply -var='worker_count=8'. Stack boots in ~90 s with custom image.

5

Run Simulation

SSH → head → mpirun --hostfile hostfile -np 32 my_app ...
Checkpoint to Swift.

6

Archive Results

swift upload sim-outputs ./results/ — results persist independent of cluster.

7

Analyse & Visualise

Post-process on ADA Jupyter or download to HPC. Publication-quality plots.

8

Clean Up

terraform destroy. openstack image delete (if obsolete). git push final state.

Steps 1,4,8 = IaC (Part 1) | Steps 2,7 = Glance/Jupyter | Steps 3,6 = Swift | Step 5 = cluster execution (both parts)

OpenStack vs AWS: Complete Service Mapping

Concepts are identical — vendor APIs differ. Skills transfer.

Category	OpenStack	AWS Equivalent	Core Concept
Identity & Auth	Keystone	IAM	Authentication + authorisation
Compute	Nova	EC2	Virtual machines
Networking	Neutron	VPC + ELB	Software-defined networking
Block Storage	Cinder	EBS	Persistent attached volumes
Object Storage	Swift	S3	Durable object store (key-value)
Image Registry	Glance	AMI (EC2 Images)	VM image catalog
Orchestration	Heat (HOT)	CloudFormation	Declarative IaC + stack lifecycle
Multi-cloud IaC	Terraform (OS provider)	Terraform (AWS provider)	Same HCL; same state model
Dashboard	Horizon	AWS Management Console	Web UI for resource management
Metadata API	Nova metadata API	EC2 Instance Metadata	Access instance context from inside VM

Mastering OpenStack teaches you the principles that underlie every cloud platform. The APIs change; the concepts are universal.

Part 2 — Key Takeaways

Custom Images

Install once, boot many times. Snapshot → versioned image → 90-second cluster launch. Version semantically; store build scripts in Git.

Swift = Durable Layer

S3-equivalent on OPENSTACK. Persists independent of VM and stack lifecycle. Primary mechanism for ADA ↔ HPC data exchange.

NFS Pattern

Cinder-backed volume on head node exported via NFS. Simple shared workspace for params and checkpoints — simplified Lustre for small clusters.

3 Integration Patterns

Interactive analysis (Jupyter), Pre/post-processing (ADA flanks HPC), Cloud bursting (embarrassingly parallel overflow only).

The Boundary is InfiniBand

ADA Cloud complements HPC — does not replace it. Tightly-coupled MPI requiring InfiniBand cannot be burst to OpenStack VMs.

Debug Systematically

① Quota → ② Stack/state → ③ Console log → ④ Network → ⑤ noVNC. Quota exhaustion is the most common silent failure mode.

Complete OpenStack Lecture Recap: Fundamentals + Advanced

What you can now do on CINECA ADA Cloud

Fundamentals (Lecture 1)

- Authenticate with Keystone using Application Credentials
- Provision VMs, networks, routers, SGs, floating IPs via CLI
- Attach and manage Cinder persistent block volumes
- Automate instance configuration with cloud-init / user_data
- Understand the full OpenStack ↔ AWS service equivalence map
- Debug failures using console log and SG inspection

Advanced (Lecture 2)

- Write Heat HOT templates with ResourceGroup for multi-node stacks
- Write Terraform HCL with OpenStack provider; manage state safely
- Choose Heat vs Terraform by operational context
- Build custom HPC images with Glance; version and document
- Stage data to/from Swift for durable HPC-cloud exchange
- Design hybrid HPC-cloud workflows across ADA and Leonardo

References and Further Reading

Official Documentation

- Heat (Wallaby): docs.openstack.org/heat/wallaby/
- HOT Spec:
docs.openstack.org/heat/wallaby/template_guide/hot_spec.html
- TF OpenStack provider: registry.terraform.io/providers/terraform-provider-openstack
- Swift: docs.openstack.org/swift/wallaby/
- Glance: docs.openstack.org/glance/wallaby/
- CINECA ADA: wiki.u-gov.it/confluence/display/SCAIUS/ADA+Cloud
- HashiCorp Terraform: developer.hashicorp.com/terraform/docs

Books, Papers, Related Lectures

- Brikman (2022). Terraform: Up & Running (3rd ed.). O'Reilly. — State management + modules
- Sefraoui et al. (2012). OpenStack: Toward an Open-Source Solution for Cloud Computing. Int. J. Computer Applications 55(3).
- Kleppmann (2017). Designing Data-Intensive Applications. O'Reilly. — Eventual consistency in object stores
- OpenStack User Survey 2023: openstack.org/user-survey/
- L02: Cloud Economics — pricing, FinOps
- L04: Docker images — same versioning principles
- L05: Kubernetes — same declarative control loop
- L08: Terraform (AWS) — same HCL applied here

Questions?